



Security Review For Morpho



Collaborative Audit Prepared For:
Lead Security Expert(s):

Morpho
0x73696d616f
pkqs90
xiaoming90

Date Audited:

September 22 - September 23, 2026

Introduction

MidnightBundlesV2 contract provides end-user entrypoints for the Midnight fixed-rate lending protocol. It iterates on the MidnightBundlesV1 contract. On the maker side, a new entrypoint lets makers update their offers in a single transaction: cancel existing offers, fund new ones with collateral or with loan assets supplied to Morpho Blue, and ratify and publish a new set of offers. On the taker side, sell functions first withdraw available credit before taking offers, and buy functions can repay remaining debt after taking offers, allowing positions to be fully exited. Finally, all functions now support native tokens and a deadline, while permit-based approvals are removed.

Scope

Repository: [morpho-org/bundles](https://github.com/morpho-org/bundles)

Audited Commit: [4f7ea167ce30957ee677231cb73dae987fba7757](https://github.com/morpho-org/bundles/commit/4f7ea167ce30957ee677231cb73dae987fba7757)

Final Commit: [2ed954e51c98fcfffd17ab6c596f5998e6b3e321](https://github.com/morpho-org/bundles/commit/2ed954e51c98fcfffd17ab6c596f5998e6b3e321)

Files:

- `src/midnight/interfaces/IMidnightBundlesV2.sol`
- `src/midnight/MidnightBundlesV2.sol`

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	12

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Incorrect wrapping unit assumption [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/3>

Vulnerability Detail

The `wrapNativeToMsgSender()` function within various bundle functions allows the caller to wrap native assets to their chosen wrapper.

```
File: MidnightBundlesV2.sol
77:     function midnightBundlesV2CancelAndMake(
    ..SNIP..
95:     ) external payable {
96:         require(block.timestamp <= deadline, DeadlinePassed());
97:         if (msg.value > 0) wrapNativeToMsgSender(wrappedNative);
```

```
File: MidnightBundlesV2.sol
458:     /// @dev Wraps msg.value into wrappedNative and transfers it to msg.sender.
459:     // forge-lint: disable-next-item(arbitrary-send-eth) wrappedNative is
    ↪ chosen by msg.sender, who also receives the wrapped tokens.
460:     function wrapNativeToMsgSender(address wrappedNative) internal {
461:         IWNative(wrappedNative).deposit{value: msg.value}();
462:         SafeTransferLib.safeTransfer(wrappedNative, msg.sender, msg.value);
463:     }
```

In the `wrapNativeToMsgSender` function, it first transfers `msg.value` units of the native asset to the wrapper. Then, it transfers `msg.value` units of the wrapped asset to the caller.

The current design assumes that the native asset's unit and the wrapped asset's unit are always the same. However, this assumption, or invariant, does not always hold on every current or future supported chain.

In addition, the caller can define `wrappedNative` arbitrarily. As a result, the caller can point it to a third-party wrapper instead of the chain's canonical wrapper with a different unit, increasing risk.

For instance, on Arc, its native token is USDC, and its native interface has 18 decimals, while its ERC-20 interface has 6 decimals. Although Arc requires no wrapping, this illustrates that the units of the native token and its wrapper counterpart are not always the same.

Another example is YBTC.B on Bitlayer. Native BTC uses 18 decimals, while YBTC.B uses 8. Thus, the following issue will occur:

1. User sends 1 native BTC (`msg.value = 1e18`)

2. `deposit()` credits the bundle `1e8 YBTC.B` units
3. `safeTransfer()` attempts to send `1e18 YBTC.B` units
4. Revert due to insufficient balance

Impact

The assumed 1:1 base-unit conversion can cause transactions to revert when fewer wrapped units are minted, or strand excess tokens in the bundle when more are minted.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L460

Tool Used

Manual Review

Recommendation

Consider transferring the token balance increase measured around `deposit()`, instead of transferring `msg.value`. Alternatively, explicitly document that native wrapping supports only wrappers that mint one token base unit per native base unit.

Discussion

Morpho

Fixed in <https://github.com/morpho-org/bundles/pull/177>

Issue L-2: midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget function may fail on valid offers due to division-by-zero error [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/4>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget()` sizes each offer outside its `try/catch`. A positive-tick buy offer whose price is equal to the current settlement fee will cause a division-by-zero panic before `Midnight.take()` is reached, even though the same offer is accepted by `Midnight.take()`.

One such offer prevents the bundle from trying later liquidity that can satisfy the user's target.

Root cause

The for loop in `midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget` converts its remaining asset target into units for each offer. It calls `TakeAmountsLib.sellerAssetsToUnits()` before the external `Midnight.take()` call covered by `try/catch`:

Source: `midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget()`

```
for (uint256 i; i < offerFills.length && filledSellerAssets <
↳ targetFilledSellerAssets; i++) {
    ...
    // audit-bug: this offer-specific calculation can revert before the try/catch.
    uint256 unitsToTake = min(
        TakeAmountsLib.sellerAssetsToUnits(
            MIDNIGHT, id, fill.offer, targetFilledSellerAssets - filledSellerAssets
        ),
        fill.units,
        ConsumableUnitsLib.consumableUnits(MIDNIGHT, id, fill.offer)
    );
    ...
    @> try IMidnight(MIDNIGHT)
        .take(fill.offer, fill.ratifierData, unitsToTake, msg.sender,
        ↳ address(this), address(0), "")
    returns (uint256, uint256 resSellerAssets) {
        filledUnits += unitsToTake;
        filledSellerAssets += resSellerAssets;
```

```

    } catch {}
}

```

For a buy offer, the helper subtracts the current settlement fee from the offer price and uses the result as a divisor. Equality makes `sellerPrice` zero, so `mulDivUp()` panics:

Source: [TakeAmountsLib.sellerAssetsToUnits\(\)](#)

```

function sellerAssetsToUnits(address midnight, bytes32 id, Offer memory offer,
    ↪ uint256 targetSellerAssets)
    internal
    view
    returns (uint256)
{
    uint256 offerPrice = TickLib.tickToPrice(offer.tick);
    uint256 settlementFee =
        IMidnight(midnight).settlementFee(id,
            ↪ UtilsLib.zeroFloorSub(offer.market.maturity, block.timestamp));
    uint256 sellerPrice = offer.buy ? offerPrice - settlementFee : offerPrice;
    // audit-note: a positive target is divided by sellerPrice == 0.
    return
        offer.buy ? targetSellerAssets.mulDivUp(WAD, sellerPrice) :
            ↪ targetSellerAssets.mulDivDown(WAD, sellerPrice);
}

```

In comparison, the `Midnight.take()` function actually accepts `offerPrice == settlementFee`, calculates `sellerPrice == 0`, and returns zero seller assets.

Source: [Midnight.take\(\)](#)

```

uint256 offerPrice = TickLib.tickToPrice(offer.tick);
uint256 timeToMaturity = UtilsLib.zeroFloorSub(offer.market.maturity,
    ↪ block.timestamp);
uint256 _settlementFee = settlementFee(id, timeToMaturity);
uint256 sellerPrice = offer.buy ? offerPrice - _settlementFee : offerPrice;
uint256 buyerPrice = sellerPrice + _settlementFee;
uint256 buyerAssets = offer.buy ? units.mulDivDown(buyerPrice, WAD) :
    ↪ units.mulDivUp(buyerPrice, WAD);
uint256 sellerAssets = offer.buy ? units.mulDivDown(sellerPrice, WAD) :
    ↪ units.mulDivUp(sellerPrice, WAD);

```

Because the panic occurs in the bundle's own frame, the `try/catch` around `Midnight.take()` cannot catch it and continue to the next offer.

Impact

The caller loses the transaction's gas and receives no output, even though the route contains enough valid fallback liquidity.

Discussion

Morpho

We acknowledge this issue. Note that it was pre-existing on the midnight bundles v1, and that this is a liveness issue that can be mitigated by changing the arguments of the functions (lowering the fill amount, or removing offers).

To give more details, here are the scenarios in which the `TakeAmountsLib` functions could revert when called in the bundles:

1. if `buyerPrice > WAD` in `buyerAssetsToUnits`. Note that this can't happen in the case where the buyer is the maker, because the `tickToPrice` function never gives prices $> WAD$. So we consider the case where the taker is the buyer. But the taker would probably prefer that the bundles revert in that case: it makes no sense for buying new credit (negative rate) and neither for repaying debt (always possible to have something better with `repayEnabled`). Prices too close to WAD for the taker-buyer should be filtered out by the router.
2. `offer.buy` and `settlementFee >= offerPrice`. Then it reverts for underflow or potentially division by 0. Note that if the offers are sorted by price then as soon as `settlementFee >= offerPrice` for one offer, that stays true for following offers. So even if the iteration is skipped, the function would eventually revert. Also it would mean that the taker is selling at a loss for these offers. Prices too close to 0 for the taker-seller should be filtered out by the router.
3. price is 0 which produces a division by 0. Because of case 2, we can restrict ourselves to `!offer.buy`. In that case the offer price is 0 (looking at what `TakeAmountsLib` computes). The rate is infinite here, which also isn't very likely except if the maker-seller wants to transfer its credits. It's ok if this is not supported by the bundler, this can be done on Midnight directly if needed. Offer price 0 to sell should be filtered out by the router.

As a summary: the cases where one take-iteration reverts are bad cases for the taker, or unsupported. The router could & should take some buffer to ensure to not hit those, and the bundler reverts if the buffer is not enough

Issue L-3: The payloadToLog field in MidnightBundlesV2::midnightBundlesV2CancelAndMake() is called payload in the interface [RESOLVED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/5>

Summary

IMidnightBundlesV2::midnightBundlesV2CancelAndMake() declares the argument payload, while MidnightBundlesV2::midnightBundlesV2CancelAndMake() calls it payloadToLog.

Vulnerability Detail

In IMidnightBundlesV2::midnightBundlesV2CancelAndMake() the field is called payload:

```
GroupCancellation[] memory groupsToCancel,  
bytes memory payload,  
uint256 deadline,
```

But in MidnightBundlesV2::midnightBundlesV2CancelAndMake(), it is payloadToLog:

```
GroupCancellation[] memory groupsToCancel,  
bytes memory payloadToLog,  
uint256 deadline,
```

Impact

Inconsistency.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/interfaces/IMidnightBundlesV2.sol#L63 https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L92

Tool Used

Manual Review

Recommendation

Use the same word in both files.

Discussion

Morpho

Fixed in <https://github.com/morpho-org/bundles/pull/176>

Issue L-4: Front-running liquidations can block full repayment and collateral withdrawal [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/6>

Vulnerability Detail

MidnightBundlesV2 supports repayment and collateral withdrawal through its buy functions with `repayEnabled = true` and an empty `offerFills` array. However, both functions require an exact target supplied before execution.

In Line 204, it caps the repayment to the borrower's current debt, then requires `filledUnits == targetUnits` in Line 210 below.

```
File: MidnightBundlesV2.sol
159:     function midnightBundlesV2BuyWithUnitsTargetAndWithdrawCollateral(
    ..SNIP..
203:         if (repayEnabled) {
204:             uint256 repayUnits = UtilsLib.min(targetUnits - filledUnits,
    ↪ IMidnight(MIDNIGHT).debt(id, msg.sender));
205:             IMidnight(MIDNIGHT).repay(market, repayUnits, msg.sender,
    ↪ address(0), "");
206:             filledUnits += repayUnits;
207:             filledBuyerAssets += repayUnits;
208:         }
209:
210:         require(filledUnits == targetUnits, OutOfOffers());
```

Consequently, a reduction in debt before execution causes the bundle to revert even though it could fully repay the remaining debt.

Consider the following scenario:

1. A borrower with debt D submits a full repayment and collateral withdrawal, using target D , no offers, and zero referral fee.
2. The attacker front-runs with a one-unit liquidation, reducing the borrower's debt from D to $D - 1$
3. The bundle repays $D - 1$, then reverts with `OutOfOffers()` because the original target remains D
4. The attacker can repeat when the borrower resubmits with an updated target

Note: After maturity, even healthy borrowers are liquidatable if the liquidator gate permits the attacker and the other liquidation conditions are satisfied.

Impact

An attacker can repeatedly cause full repayments and collateral withdrawals to revert, wasting the borrower's gas and delaying position closure. The remaining debt stays exposed to further liquidation during the delay.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L204

Tool Used

Manual Review

Recommendation

Considering an explicit full-close mode that resolves repayment and collateral withdrawal amounts from live balances at execution time.

Discussion

Morpho

Fixed in <https://github.com/morpho-org/bundles/pull/174>

Issue L-5: The referral fee is not reserved before the take loop in MidnightBundlesV2::midnightBundlesV2BuyWithUnitsTargetAndWithdrawCollateral() [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/7>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

MidnightBundlesV2::midnightBundlesV2BuyWithUnitsTargetAndWithdrawCollateral() computes the referral fee after the take loop has already spent `maxBuyerAssets`, with no cap. The units to take should take into account that a referral fee will be paid out.

Vulnerability Detail

MidnightBundlesV2::midnightBundlesV2BuyWithUnitsTargetAndWithdrawCollateral() states that `msg.sender` will pay at most `maxBuyerAssets`, and the units to take are capped in:

```
uint256 unitsToTake = min(
    targetUnits - filledUnits, fill.units,
    ↳ ConsumableUnitsLib.consumableUnits(MIDNIGHT, id, fill.offer)
);
```

Midnight::take() pulls `buyerAssets` from MidnightBundlesV2, so as long as it holds enough assets and units to take are not reached, the loop will continue. This means that after the loop, `maxBuyerAssets` may have been exhausted, but the fee is then transferred, which will revert.

MidnightBundlesV2 says the taker that an offer it cannot fill is skipped so the rest of offer `Fills` still fills. This is not always true, for example:

`targetUnits = 1000e18`, `referralFeePct = 0.01e18` and two sell offers in `offerFills`, costing `1000e18` and `500e18` each:

1. The taker sets `maxBuyerAssets = 1000e18`.
2. The loop reaches the first offer, takes the whole `1000e18` units for `1000e18` assets, and exits with `filledUnits == targetUnits`. The second offer not reached.
3. `referralFeeAssets` is $1000e18 * 0.01e18 / 0.99e18 = 10.101e18$ and the contract holds 0, so the transfer to `referralFeeRecipient` reverts.

Taking the second offer instead would have cost $500e18$ plus a $5.05e18$ fee, which would succeed since max buyer assets is $1000e18$, and the $1000e18$ offer would be skipped (catch).

The three other functions behave correctly:

- `midnightBundlesV2BuyWithAssetsTargetAndWithdrawCollateral()` subtracts the fee into `targetFilledBuyerAssets` and then caps every offer by what is left of it;
- `midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget()` does the same;
- `midnightBundlesV2SupplyCollateralAndSellWithUnitsTarget()` takes the fee out of assets it has already received.

Impact

Any take through `midnightBundlesV2BuyWithUnitsTargetAndWithdrawCollateral()` with a non-zero `referralFeePct` reverts whenever the fill exceeds the `referralFeePct`.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L191

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L223-L227

Tool Used

Manual Review

Recommendation

Handle the fee instead of reverting due to out of funds, or document this explicitly.

Discussion

Morpho

MidnightBundlesV2 says the taker that an offer it cannot fill is skipped so the rest of `offerFills` still fills.

The exact comment is

The buy/sell functions below skip the offer if the take reverted. This avoids reverting the whole call when other offers passed as argument still have liquidity.

which stays true even in the highlighted scenario: the offer is skipped if the `take` itself reverted (but here `take` didn't revert).

What you suggest would be a nice behavior though. Still I think we can acknowledge this issue because reverting because of the fee should be rare enough, when the `maxBuyerAssets` is set properly

Morpho

We will acknowledge this issue in the end. If the offers are sorted by price, where the lowest (= best for the taker), then the current code is not creating more reverts than it should. This is because the `targetUnits` have to be met, and the cheapest way to do that is by the greedy algorithm, trying fill the units with the less costly offers first. If that's still making you pay more than `maxBuyerAssets`, so would any other order of offers, and corresponding filled amounts

Issue L-6: Duplicated supply collateral loop [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/8>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`midnightBundlesV2CancelAndMake()`, `midnightBundlesV2SupplyCollateralAndSellWithUnitsTarget()` and `midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget()` contain the same collateral supply loop, which could be moved into a helper, like it was done previously.

Vulnerability Detail

The following code is repeated 3 times:

```
for (uint256 i; i < collateralSupplies.length; i++) {
    address collateralToken =
        ↪ market.collateralParams[collateralSupplies[i].collateralIndex].token;
    SafeTransferLib.safeTransferFrom(collateralToken, msg.sender, address(this),
        ↪ collateralSupplies[i].assets);
    TokenLib.forceApproveMax(collateralToken, MIDNIGHT);
    IMidnight(MIDNIGHT)
        .supplyCollateral(
            market, collateralSupplies[i].collateralIndex,
            ↪ collateralSupplies[i].assets, msg.sender
        );
}
```

In the previous code they called `supplyCollaterals(market, collateralSupplies, msg.sender)`, passing `msg.value > 0 && i == 0` to `TokenLib::transferFromOrWrapNative()` so that only the first iteration wrapped native. This update moved native wrapping into `wrapNativeToMsgSender()`, called at the beginning of each function, and inlined the collateral supply loop at every location, which could stay in a helper.

Impact

Duplicated code.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L108-L116
https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L254-L262
https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L405-L413

Tool Used

Manual Review

Recommendation

Place the duplicated code inside an helper.

Discussion

Morpho

We acknowledge this issue, depending on the case it might be better to inline or not. It was discussed in [this thread](#)

Issue L-7: MidnightBundlesV2 doesn't emit the referral fee nor the recipient [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/9>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

MidnightBundlesV2 emits no events, which is fine for most operations since they are emitted externally, for example in Midnight itself. However, `referralFeePct` and `referralFeeRecipient` are not emitted anywhere.

Vulnerability Detail

`Midnight::take()` emits most information of the take per fill, but not the referral fee and its recipient.

The only trace of `referralFeeAssets` is the transfer, which is not very explicitly and most importantly doesn't say who paid the fee.

Impact

Reconstructing the referral fees paid offchain might be difficult.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L223-L224

Tool Used

Manual Review

Recommendation

Declare the event on `IMidnightBundlesV2` and emit it wherever the referrer is paid, for example:

```
event ReferralFee(  
    address indexed caller,  
    bytes32 indexed marketId,
```

```
    address indexed referralFeeRecipient,  
    address token,  
    uint256 referralFeePct,  
    uint256 assets  
);
```

```
-         if (referralFeeAssets > 0) SafeTransferLib.safeTransfer(loanToken,  
↪ referralFeeRecipient, referralFeeAssets);  
+         if (referralFeeAssets > 0) {  
+             SafeTransferLib.safeTransfer(loanToken, referralFeeRecipient,  
↪ referralFeeAssets);  
+             emit ReferralFee(msg.sender, id, referralFeeRecipient, loanToken,  
↪ referralFeePct, referralFeeAssets);  
+         }
```

Discussion

Morpho

We don't have events in this repository, rationale being that it has no state: the meaningful state that you may want to track is on the underlying protocols used. For transfers, the relevant state is also in the tokens themselves. Tracking might indeed be simpler with some events, but it does not seem impossible without either

Issue L-8: MidnightBundlesV2::midnightBundlesV2CancelAndMake() still reverts when the same group is listed twice in groupsToCancel [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/10>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

MidnightBundlesV2::midnightBundlesV2CancelAndMake() cancels a group by writing `type(uint128).max` into Midnight's consumed mapping. If there is a repeated group it reverts unless `cancellation.maxConsumed` was already `type(uint128).max`. There used to be a comment acknowledging this behaviour but it was removed.

Vulnerability Detail

In `MidnightBundlesV2::midnightBundlesV2CancelAndMake()`, note how it reverts if groups are repeated, as `IMidnight(MIDNIGHT).consumed(msg.sender, cancellation.group)` will be `type(uint128).max` for the second iteration, possibly higher than `cancellation.maxConsumed`.

```
for (uint256 i; i < groupsToCancel.length; i++) {
    GroupCancellation memory cancellation = groupsToCancel[i];
    require(
        IMidnight(MIDNIGHT).consumed(msg.sender, cancellation.group) <=
        ↪ cancellation.maxConsumed, //@audit reads what the previous iteration
        ↪ wrote
        ConsumedAboveMax()
    );
    IMidnight(MIDNIGHT).setConsumed(cancellation.group, type(uint128).max,
    ↪ msg.sender);
}
```

The previous natspec comment about this revert was removed:

```
/// @dev After a group is cancelled, later occurrences of its ID in groupsToCancel
↪ revert unless their maxConsumed is type(uint128).max.
```

Impact

Repeated groups in the same cancellation call revert the whole transaction.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/main/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L99-L106

Tool Used

Manual Review

Recommendation

Skip a group that the loop has already cancelled or keep the old natspec.

Discussion

Morpho

We acknowledge the issue, as we feel that it is straightforward enough and only a liveness issue that can be solved by handling the cancels properly. See [this thread](#)

Issue L-9: Resting borrow offers can block full repayment and collateral withdrawal [RESOLVED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/12>

Vulnerability Detail

The new `repayEnabled` paths in the `MidnightBundlesV2` buy functions cap repayment at the caller-specified target, even when the borrower's live debt has increased. A front-running fill of an outstanding borrow-limit sell offer can therefore leave residual debt and prevent a full collateral withdrawal.

Consider the following scenario:

1. Before maturity, a borrower has debt D , no credit, and a valid resting sell offer with `reduceOnly = false`. The offer permits the attacker to take it, and the borrower has enough collateral to support another debt unit.
2. The borrower submits a full close with `targetUnits = D`, `repayEnabled = true`, an empty `offerFills` array, zero referral fee, and withdrawal of all collateral.
3. The attacker front-runs by buying one unit from the resting offer. Provided the other take checks pass, the borrower's debt increases to $D + 1$.
4. The bundle repays D and passes its exact-target check, but one debt unit remains. Withdrawing all collateral reduces the borrowing capacity to zero, below the remaining debt. Midnight's health check therefore reverts with `UnhealthyBorrower()`, rolling back the repayment and withdrawal.

The `assets-target` buy function has the same failure when `targetBuyerAssets = D` and the referral fee is zero.

Impact

An attacker can delay repayment and collateral recovery by filling outstanding borrow offers before close attempts, wasting the borrower's gas and keeping the position open.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L203-L221

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L355-L375

Tool Used

Manual Review

Recommendation

Consider canceling or revoking resting borrow offers before attempting a full close, and resolve repayment from live debt within the caller's spending limit.

Discussion

Morpho

Acknowledged. The user is aware of both the resting offer and the current transaction. They behave as expected:

- the offer can be taken at any time, increasing the debt
- the repay transaction tries to repay an exact amount (notably the function name doesn't say "full close")

Morpho

We added repayment from live debt in

<https://github.com/morpho-org/bundles/pull/174>, to mitigate the scenario where the borrower could be liquidated (notably after maturity). It is also a fix for this issue

Issue L-10: Fee accrual or slashing can cause reduce-only maker sell offers to be skipped in full [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/13>

Vulnerability Detail

The four take loops cap `unitsToTake` by the remaining target, `fill.units`, and unconsumed offer capacity, without accounting for the maker's live position.

For a reduce-only sell offer, a fill sized to the maker's full effective credit at simulation can exceed its credit at inclusion once additional fees accrue. `Midnight.take()` refreshes the position and reverts with `MakerCreditOrDebtIncreased`. The bundle catches the revert and skips the entire offer, including its executable portion.

```
File: MidnightBundlesV2.sol
191:         uint256 unitsToTake = min(
192:             targetUnits - filledUnits, fill.units,
    ↪ ConsumableUnitsLib.consumableUnits(MIDNIGHT, id, fill.offer)
193:         );
..SNIP..
195:         try IMidnight(MIDNIGHT)
196:             .take(fill.offer, fill.ratifierData, unitsToTake, msg.sender,
    ↪ address(0), address(0), "") returns (
..SNIP..
201:         } catch {}
..SNIP..
210:         require(filledUnits == targetUnits, OutOfOffers());
```

Consider a buy route before maturity, using a six-decimal loan token, with `repayEnabled = false` and `taker reduceOnly = false`. The amounts below are expressed in tokens. Both offers otherwise satisfy the execution checks:

1. The target is 1,000 units. Alice has 1,000 effective credit and a reduce-only sell offer with 1,000 unconsumed units; the route sets her `fill.units = 1,000`. Bob provides a backup offer for 20 units.
2. Before inclusion, Alice accrues one additional base unit of fees, reducing her credit to 999.999999. Her offer consumption is unchanged, and Bob's 20-unit offer remains executable.
3. The bundle attempts 1,000 from Alice and skips her offer after the reduce-only revert. Bob fills 20, but the remaining target is unmet, so the transaction reverts with `OutOfOffers`.

Taking 999.999999 from Alice and 0.000001 from Bob would satisfy the same target. A one-base-unit shortfall therefore causes the route to discard nearly all its executable

liquidity.

Slashing can similarly reduce maker credit. In the sell entrypoints, reduced maker debt after repayment or liquidation can cause the equivalent failure for reduce-only buy offers.

Impact

Routes can revert despite sufficient executable liquidity, wasting gas and delaying exits. Requoting the full current credit can repeat the failure whenever additional fees of at least one base unit accrue before inclusion.

Code Snippet

https://github.com/sherlock-audit/2026-09-morpho-midnight-bundles-update-sep-22/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L191-L201

https://github.com/sherlock-audit/2026-09-morpho-midnight-bundles-update-sep-22/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L274-L287

https://github.com/sherlock-audit/2026-09-morpho-midnight-bundles-update-sep-22/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L339-L353

https://github.com/sherlock-audit/2026-09-morpho-midnight-bundles-update-sep-22/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L428-L445

Tool Used

Manual Review

Recommendation

When `fill.offer.reduceOnly` is true, consider additionally capping each attempted fill by the maker's current credit for sell offers or current debt for buy offers. Use `updatePositionView()` for credit so the cap includes accrued fees and slashing.

Discussion

Morpho

The front-end could always take a buffer and pass a `fill.units` a bit lower so that it would mitigate this issue. This is one of the purposes of this parameter. Still, it is easy

enough to implement, and would make it easier / more natural for the frontend. We will discuss internally and come back to you

Morpho

Fixed in <https://github.com/morpho-org/bundles/pull/179>

Issue L-11: Blue parking liquidity risk [RESOLVED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/14>

Vulnerability Detail

`midnightBundlesV2CancelAndMake()` allows makers to park loan assets on Blue under their derived `BlueBuyCallback` while waiting for Midnight offers to be taken. The assets can earn interest, but they remain available for Blue borrowers to borrow and are not reserved for the maker's offers.

When an offer is taken, `BlueBuyCallback.onBuy()` withdraws the required `buyerAssets` from the selected Blue market. If available liquidity is insufficient, the withdrawal and the Midnight take revert even when the callback owns enough supply shares to cover the amount. Ordinary borrowing can cause this condition; an attacker with sufficient collateral can also deliberately borrow before a take to prevent its execution.

`BlueBuyCallback.onBuy()` already warns that low Blue liquidity can cause fills to fail. The comment at `MidnightBundlesV2.sol:151` tells callers to choose fill sizes the callback can fund, but does not explain this liquidity risk. The NatSpec for `MidnightBundlesV2.midnightBundlesV2CancelAndMake()` and the bundles README omit this warning.

As a result, makers may assume their parked funds will always be available to fill offers.

Consider the following scenario, ignoring interest and rounding:

1. A maker parks 100,000 USDC, and the Blue market has 100,000 USDC of available liquidity.
2. A taker submits a fill requiring 80,000 USDC from the callback. Before execution, another account borrows 30,000 USDC from the same Blue market.
3. The callback still owns a 100,000-USDC supply claim, but its 80,000-USDC withdrawal reverts because only 70,000 USDC remains available.
4. In either V2 sell entrypoint, the failed take skips that offer entry without retrying a smaller fill using the available 70,000 USDC.

Impact

Makers may miss intended offer fills or face delays withdrawing parked funds, while takers may waste gas on failed transactions. Without a clear warning, users may choose Blue parking assuming their funds remain immediately available.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/mid

[night/MidnightBundlesV2.sol#L58-L76](#)

[https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L281-L290](#)

[https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L439-L448](#)

Tool Used

Manual Review

Recommendation

Consider documenting this tradeoff in the NatSpec for `MidnightBundlesV2.midnightBundlesV2CancelAndMake()`, the bundles README, and consider stating explicitly that earning interest on Blue does not guarantee Midnight offer execution or immediate withdrawal, even when the maker's supply balance is sufficient.

Discussion

Morpho

fixed in [https://github.com/morpho-org/bundles/pull/180](#)

Issue L-12: Inconsistent zero-value transfer checks in seller payouts [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/issues/15>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

MidnightBundlesV2 skips zero-value referral-fee and refund transfers, but both seller payout paths call `safeTransfer()` unconditionally.

Instance 1 - `midnightBundlesV2SupplyCollateralAndSellWithUnitsTarget()`

If a user only wants to supply collateral, they can set `targetUnits` and `minSellerAssets` to zero. No loan tokens are received, but the function still calls `safeTransfer()` with an amount of zero.

```
File: MidnightBundlesV2.sol
292:         uint256 referralFeeAssets =
    ↪ filledSellerAssets.mulDivDown(referralFeePct, WAD);
293:         require(filledSellerAssets - referralFeeAssets >= minSellerAssets,
    ↪ SellerAssetsTooLow());
294:         address loanToken = market.loanToken;
295:         if (referralFeeAssets > 0) SafeTransferLib.safeTransfer(loanToken,
    ↪ referralFeeRecipient, referralFeeAssets);
296:         SafeTransferLib.safeTransfer(loanToken, receiver, filledSellerAssets -
    ↪ referralFeeAssets);
```

Instance 2 - `midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget()`

The payout is zero when `targetSellerAssets = 0`.

```
File: MidnightBundlesV2.sol
452:         if (referralFeeAssets > 0) SafeTransferLib.safeTransfer(loanToken,
    ↪ referralFeeRecipient, referralFeeAssets);
453:         SafeTransferLib.safeTransfer(loanToken, receiver, targetSellerAssets);
```

Impact

Zero-payout calls incur an unnecessary external token call and Transfer event. Skipping these payouts saves gas on those paths and makes zero-value handling consistent.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L292-L296

https://github.com/sherlock-audit2/2026-09-morpho-midnight-bundles-update-sep-22nd/blob/c4fc3b71d967b4a65216fefddb42274f30b3df3e/morpho-org__bundles/src/midnight/MidnightBundlesV2.sol#L451-L453

Tool Used

Manual Review

Recommendation

Consider skipping each payout when its amount is zero.

```
uint256 referralFeeAssets = filledSellerAssets.mulDivDown(referralFeePct, WAD);
+ uint256 sellerAssets = filledSellerAssets - referralFeeAssets;
- require(filledSellerAssets - referralFeeAssets >= minSellerAssets,
↪ SellerAssetsTooLow());
+ require(sellerAssets >= minSellerAssets, SellerAssetsTooLow());
address loanToken = market.loanToken;
if (referralFeeAssets > 0) SafeTransferLib.safeTransfer(loanToken,
↪ referralFeeRecipient, referralFeeAssets);
- SafeTransferLib.safeTransfer(loanToken, receiver, filledSellerAssets -
↪ referralFeeAssets);
+ if (sellerAssets > 0) SafeTransferLib.safeTransfer(loanToken, receiver,
↪ sellerAssets);
```

```
- SafeTransferLib.safeTransfer(loanToken, receiver, targetSellerAssets);
+ if (targetSellerAssets > 0) SafeTransferLib.safeTransfer(loanToken, receiver,
↪ targetSellerAssets);
```

Discussion

Morpho

We acknowledged this issue. It is functionally the same, since no-ops are assumed to not revert on tokens, so it's only about gas. Here it wouldn't lead to gas savings in normal use cases, since the purpose of the sell functions is to receive assets in exchange. It is not the same story for fees, since a fee of 0 is normal behavior

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.